Writing SQL Functions — Types, Formatting & Edge Cases

MEMORIZE THIS

The Skeleton

```
CREATE OR REPLACE FUNCTION f(arg VARCHAR(50))
RETURNS DECIMAL(5,2) AS $$      -- return TYPE
DECLARE
  d_id INT;  hi NUMERIC;  lo NUMERIC;
BEGIN
  -- look up, compute, decide
  RETURN value;                 -- every path!
END;
$$ LANGUAGE plpgsql;
```

- Header = name, **params (name + type)**, `RETURNS` type.
- Body between `$$ … $$`; logic (IF/DECLARE/RETURN) needs **LANGUAGE plpgsql**.
- **OR REPLACE** redefines; `DROP FUNCTION f(varchar);` removes.

VARIABLE TYPES & FORMATTING

Types & Formatting

type	holds / use
`VARCHAR(n)`	text ≤ n chars (names)
`INT`	whole number (ids, counts)
`DECIMAL(p,s)`	p digits, s after point (money, %)
`NUMERIC`	exact number for math locals
`BOOLEAN`	TRUE / FALSE / NULL

- **DECIMAL(5,2)** = 5 digits, 2 after the point → max **999.99**. DECIMAL(10,2) → 99,999,999.99.
- Fit the scale with **`ROUND(x, 2)`** before returning.
- Match the **argument type to the spec exactly** (e.g. `dept_name VARCHAR(50)`).
- Casts: `x::numeric`, `x::int`, `x::text`; dates `to_char(d,'YYYY-MM-DD')`.

READ THE DATA

SELECT INTO & Aggregates

```
SELECT id INTO d_id
  FROM departments WHERE name = dept_name;

SELECT MAX(salary), MIN(salary) INTO hi, lo
  FROM employees WHERE department_id = d_id;
```

Over an **empty set**:

aggregate	empty →
`COUNT(*)`	**0** (never NULL)
`MAX / MIN / AVG / SUM`	**NULL**

- Capture several columns / aggregates into several vars, left to right.
- An aggregate query always returns **one row**, so `FOUND` is TRUE even when empty — detect empty with **`IS NULL`**.

THE EXAM SKILL

Edge Cases: NULL vs Sentinel

```
SELECT id INTO d_id
  FROM departments WHERE name = dept_name;
IF NOT FOUND THEN RETURN NULL; END IF; -- missing

SELECT MAX(salary), MIN(salary) INTO hi, lo
  FROM employees WHERE department_id = d_id;
IF lo IS NULL THEN RETURN -1.00; END IF;-- empty
```

- **Does it exist?** plain lookup + `IF NOT FOUND` → **NULL**.
- **Is it empty?** aggregate `IS NULL` → **sentinel** (e.g. −1.00).
- **Order:** existence first, then emptiness, then compute. (Else a missing parent wrongly hits −1.00.)
- Match the sentinel format: `-1.00`, not `-1`, for a `DECIMAL(_,2)` return.

COMPUTE & ROUND

Formulas & the INT Trap

- **% of the lower one:** `(MAX − MIN) / MIN * 100`.
- **A as % of B:** `A / B * 100`.
- **Integer-division trap:** INT/INT truncates (`30/100 = 0`). Force numeric: `* 100.0` or `::numeric`.
- Guard `IF lo = 0` if the lower value could be 0 (divide-by-zero).

```
RETURN ROUND((hi - lo) / lo * 100, 2);
-- INT columns? ->
RETURN ROUND((hi - lo) * 100.0 / lo, 2);
```

EVERY PROBLEM USES THIS

Assumed Schema

```
departments(id INT PK,
            name VARCHAR(50))

employees(id INT PK,
          first_name VARCHAR(50),
          last_name  VARCHAR(50),
          title      VARCHAR(50),
          salary     DECIMAL(10,2),
          department_id INT -> departments.id,
          manager_id    INT -> employees.id)
```

Attack order: **header** (types) → **DECLARE** → **exists?** (NULL) → **read + empty?** (sentinel) → **compute + ROUND** → **RETURN**.

Worked Problems — One per Return Type

DECIMAL · INT · VARCHAR · BOOLEAN · pitfalls

DECIMAL(5,2) · THE MODEL PROBLEM

Salary % Spread %

% diff (of the lower) between MAX and MIN salary; NULL if dept missing; −1.00 if no employees.

```
CREATE OR REPLACE FUNCTION sal_spread(
       dept_name VARCHAR(50))
RETURNS DECIMAL(5,2) AS $$
DECLARE d_id INT; hi NUMERIC; lo NUMERIC;
BEGIN
  SELECT id INTO d_id
    FROM departments WHERE name = dept_name;
  IF NOT FOUND THEN RETURN NULL; END IF;
  SELECT MAX(salary), MIN(salary) INTO hi, lo
    FROM employees WHERE department_id = d_id;
  IF lo IS NULL THEN RETURN -1.00; END IF;
  RETURN ROUND((hi - lo) / lo * 100, 2);
END;
$$ LANGUAGE plpgsql;
```

INT · COUNT

Head Count INT

Number of employees; NULL if dept missing. (Empty → 0 automatically.)

```
CREATE OR REPLACE FUNCTION headcount(
       dept_name VARCHAR(50))
RETURNS INT AS $$
DECLARE d_id INT; n INT;
BEGIN
  SELECT id INTO d_id
    FROM departments WHERE name = dept_name;
  IF NOT FOUND THEN RETURN NULL; END IF;
  SELECT COUNT(*) INTO n
    FROM employees WHERE department_id = d_id;
  RETURN n;   -- COUNT is 0, not NULL
END;
$$ LANGUAGE plpgsql;
```

Why no empty check: `COUNT(*)` is 0 for an empty dept, so 0 is already correct.

DECIMAL(10,2) · AVERAGE

Average Salary AVG

Avg salary rounded to 2 dp; NULL if dept missing; −1.00 if no employees.

```
CREATE OR REPLACE FUNCTION avg_salary(
       dept_name VARCHAR(50))
RETURNS DECIMAL(10,2) AS $$
DECLARE d_id INT; a NUMERIC;
BEGIN
  SELECT id INTO d_id
    FROM departments WHERE name = dept_name;
  IF NOT FOUND THEN RETURN NULL; END IF;
  SELECT AVG(salary) INTO a
    FROM employees WHERE department_id = d_id;
  IF a IS NULL THEN RETURN -1.00; END IF;
  RETURN ROUND(a, 2);
END;
$$ LANGUAGE plpgsql;
```

VARCHAR · STRING INFO

Employee Card TEXT

'Last, First — Title ($salary)'; 'No employee #id' if none.

```
CREATE OR REPLACE FUNCTION emp_card(emp_id INT)
RETURNS VARCHAR(120) AS $$
DECLARE ln VARCHAR(50); fn VARCHAR(50);
        ti VARCHAR(50); sal DECIMAL(10,2);
BEGIN
  SELECT last_name, first_name, title, salary
    INTO ln, fn, ti, sal
    FROM employees WHERE id = emp_id;
  IF NOT FOUND THEN
    RETURN 'No employee #' || emp_id;
  END IF;
  RETURN ln || ', ' || fn || ' — '
      || COALESCE(ti,'N/A')
      || ' ($' || sal::text || ')';
END;
$$ LANGUAGE plpgsql;
```

|| casts numbers, but any NULL → NULL; shield with **COALESCE**.

BOOLEAN

Earns Above BOOL

TRUE if salary > threshold, FALSE otherwise, NULL if employee missing.

```
CREATE OR REPLACE FUNCTION earns_above(
       emp_id INT, threshold DECIMAL(10,2))
RETURNS BOOLEAN AS $$
DECLARE sal DECIMAL(10,2);
BEGIN
  SELECT salary INTO sal
    FROM employees WHERE id = emp_id;
  IF NOT FOUND THEN RETURN NULL; END IF;
  RETURN sal > threshold;  -- already bool
END;
$$ LANGUAGE plpgsql;
```

A comparison is itself a BOOLEAN, so return it directly.

DON'T LOSE POINTS

Pitfalls & Final Check

- **FOUND to detect empty** — use `IS NULL` on the aggregate.
- **Emptiness before existence** — missing dept wrongly returns −1.00.
- **Integer division** → cast `::numeric` / `*100.0`.
- **Forgot ROUND** → overflow / too many decimals.
- **No RETURN on a path** → run-time error.
- **Type mismatch** → cast to the declared return type.

Before you submit ✔

- Arg + `RETURNS` types exact.
- Missing → NULL · empty → sentinel (right format).
- ROUND to scale · no INT division.
- Body in `$$…$$` · ends `LANGUAGE plpgsql;`